```
num_queries = 10

for i in range(num_queries):
    print("iteration", i)
    print("incumbent", train_x[train_y.argmax()], train_y.max())

    model, likelihood = fit_gp_model(train_x, train_y)

    policy = botorch.acquisition.analytic.ProbabilityOfImprovement(    ❶
        model, best_f=train_y.max()    ❶
    )    ❶

    next_x, acq_val = botorch.optim.optimize_acqf(
    ...    ❷
```

❶ Probability of Improvement policy.

❷ Omitted.

NOTE **The number of function evaluations in Bayesian optimization**

The number of queries we use in Bayesian optimization entirely depends on how many function evaluations we can afford to make. Section 1.1 of Chapter 1 defines the problem of expensive, black-box optimization, which assumes that the number of queries we can make is relatively low due to the cost of function evaluation.

We use ten queries to run the Bayesian optimization policies for the one-dimensional Forrester function and inspect the behavior of the policies. Exercise 2 of this chapter deals with a two-dimensional function and uses twenty queries.

Figure 4.13 shows these plots at the first, fifth, and last iterations. We see that the PoI policy stays inside the region between 0 and 2, and at the tenth and last iteration, we have converged at a local optimum. This means that we are failing to adequately explore the search space and therefore missing out on the global optimum of the objective function on the right area around 4.

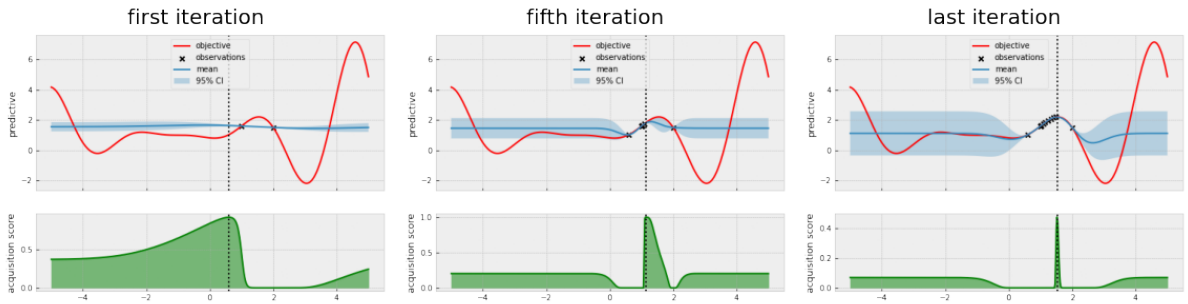


Figure 4.13 Progress made by the Probability of Improvement policy. As the policy seeks to pursue improvement of any magnitude, progress gets stuck at the local optimum near 2, and we fail to explore other regions of the search space.

©Manning Publications Co. To comment go to liveBook
https://livebook.manning.com/#!/book/bayesian-optimization-in-action/discussion

Licensed to ashwin sampath <ashwin.manning@gmail.com>